

GIT & GITHUB

GLY606 Water Data Analysis & Modeling

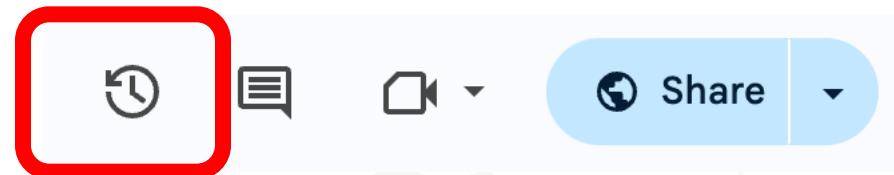
Oct 30th 2024



Homework #6

- Each team will only need to submit one homework
 - **Confirm that you all receive the shapefile for your selected basins.**
 - Find all available USGS sites available in the basin.
 - Plot a domain map for this basin with the location of all USGS sites on it
 - Conduct exploratory data analysis (EDA) using the USGS data
 - Write a two-page report
 - 1. Briefly introduce this river basin and why does your team pick this river basin?
 - 2. Summarize the EDA (Did you find any interesting trend in mean annual flow or peak flow?)
 - Summarize the information for all USGS sites into a table and attach it in the appendix (This information is not counted towards the two-page limit)
 - Information required for each site: site name, site ID, latitude, longitude, confluence area, data availability (start year, end year), etc.

Version control is everywhere!



Google documents

Version control is everywhere!

Cloud computing

EBS Snapshots

- Make a backup (snapshot) of your EBS volume at a point in time
- Not necessary to detach volume to do snapshot, but recommended
- Can copy snapshots across AZ or Region





git

≠



GitHub

About Git

- Created by Linus Torvalds, creator of Linux, in 2005
 - Came out of Linux development community
 - Designed to do version control on Linux kernel
- Goals of Git:
 - Speed
 - Support for non-linear development (thousands of parallel branches)
 - Fully distributed
 - Able to handle large projects efficiently



git



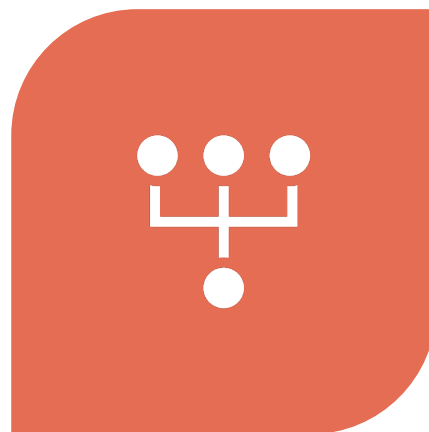
First time setup

- `git config --global user.name "Yifan Cheng"`
- `git config --global user.email ycheng46@buffalo.edu`
 - This email should be registered in your Github (more on this later)
- `git config --list --show-origin`
 - Use this command to check whether your name and email are set up correctly

Three use cases



#1: HISTORY OF
VERSIONS



#2: BRANCHING



#3: COLLABORATION
(GITHUB)

Use case #1: history of versions

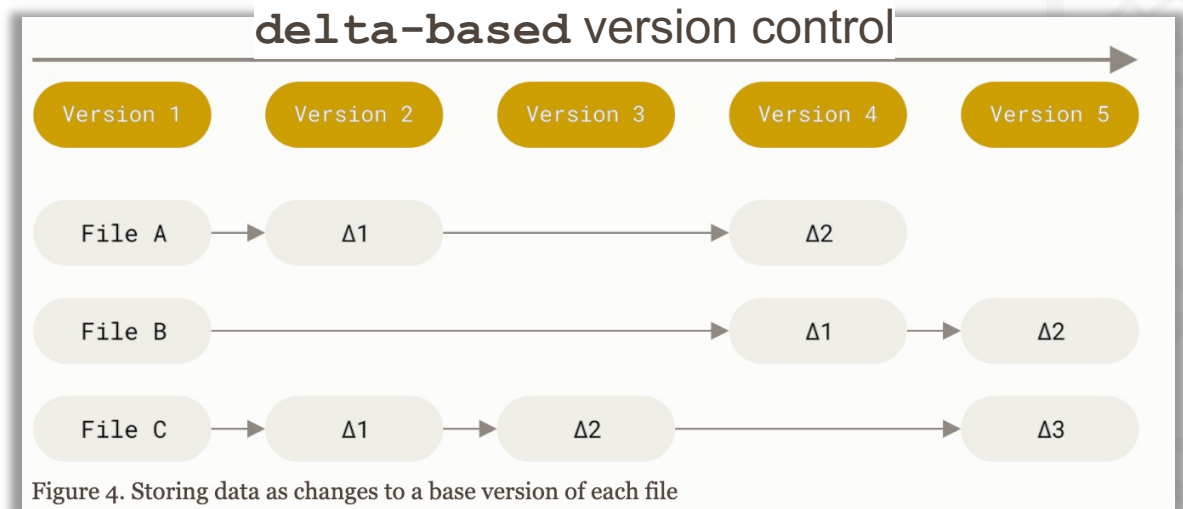
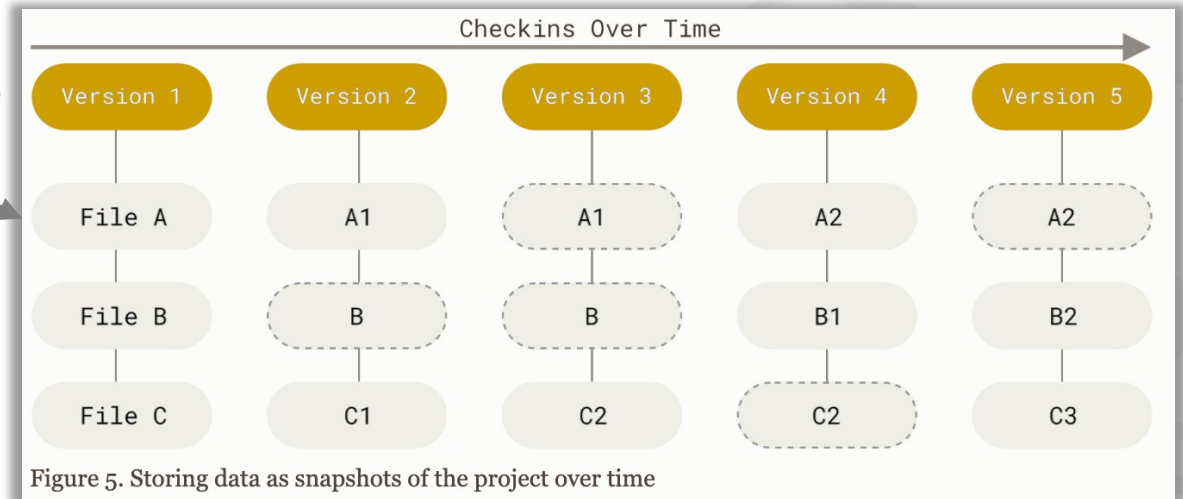
- Basic workflow
- **Adding** and **committing** files
- The git log
- The **staging** area
- **Removing** files
- Viewing diffs of files
- The .gitignore file



Big Ideas

- Snapshots, not deltas
- Everything is confined to the .git directory
- Most operations are safe – they only add data
- 3 possible states for a file
 - Modified
 - Staged
 - Committed

Git uses snapshots

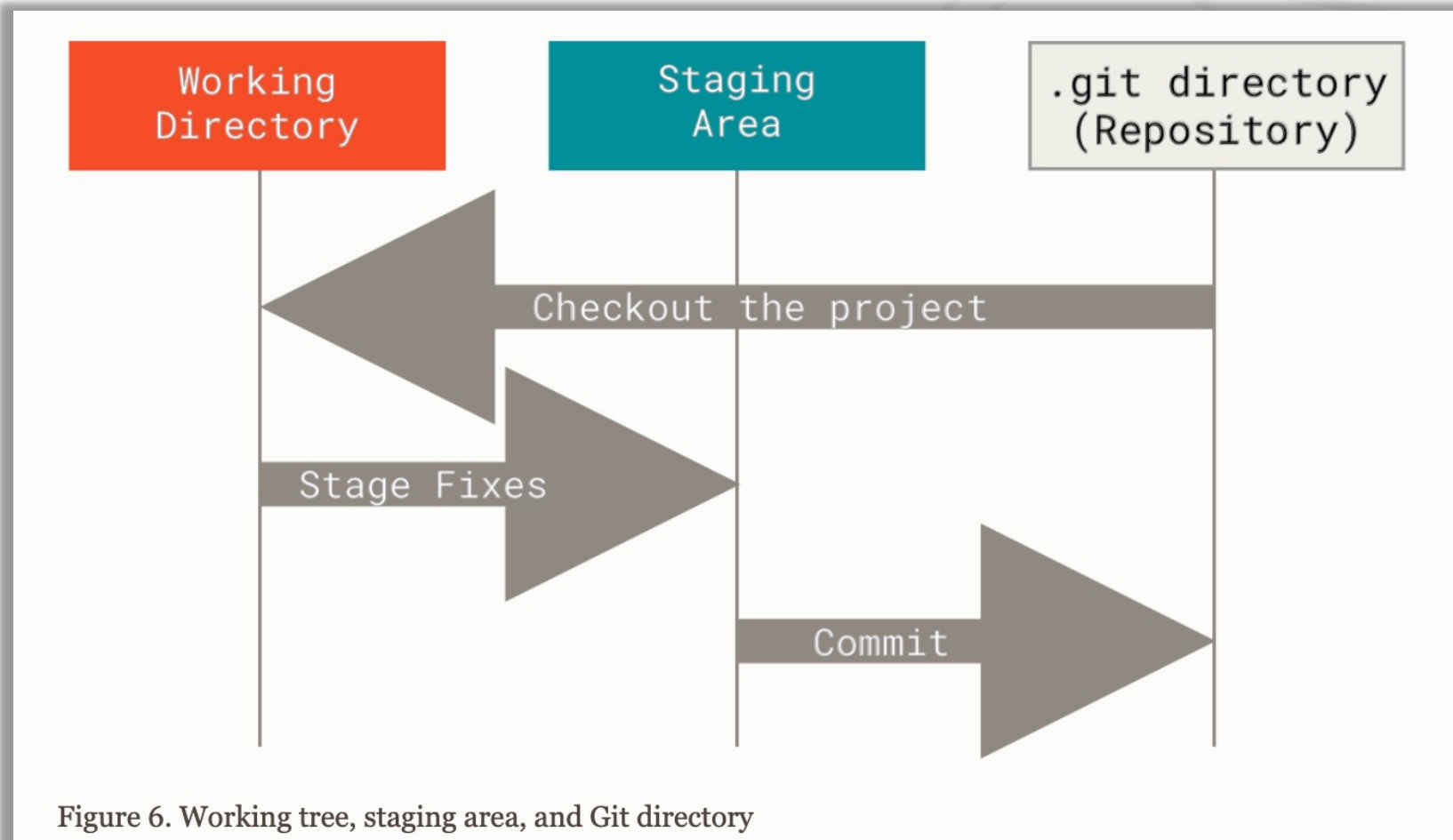


Basic workflow

- **git init** – create git project in existing directory
 - Make **Git** start to “watch” for changes in the directory
- The basic workflow

```
$ cd ~  
$ mkdir gittrain  
$ cd gittrain  
$ git init
```

Follow along using the command in blue box



Basic workflow

- Add files to be committed with **git add** <filename>
 - Puts the file in the “staging area”
- Create a commit (a “snapshot”) of added files with **git commit**
 - **git commit -a** will commit both staged and unstaged changes
- Use **git status** to see the current status of your working tree

Practice

```
$ touch b
$ vim b
$ git add b
$ git status
$ vim b
$ git status
```

This is another test

This is another test??

```
$ touch a
$ vim a
$ git add a
$ git status
$ vim a
$ git status
```

This is a test

This is a test??

```
$ git commit -m "commit #1"
$ git status
$ git add a # stage the file
$ git commit -m "commit #2"
$ git status
$ vim a
```

The git status output

```
(base) jovyan@jupyter-yifancheng-5fub:~/test/git_practice$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   b

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   b
```

The staging area

- To unstage a file, but retain your changes in the file

git restore --staged <file>...

- To discard current changes in the working tree, which will not affect the staged files:

git restore <file>...

- To stage the changes to a file,

git add <file>...

I will illustrate it, but do not follow



Removing a file

- To remove it from the next commit, but keep the file in the working tree, do **git rm --cached <filename>**
- To remove a file from the working tree and in the next commit, simply **git rm -f <filename>**



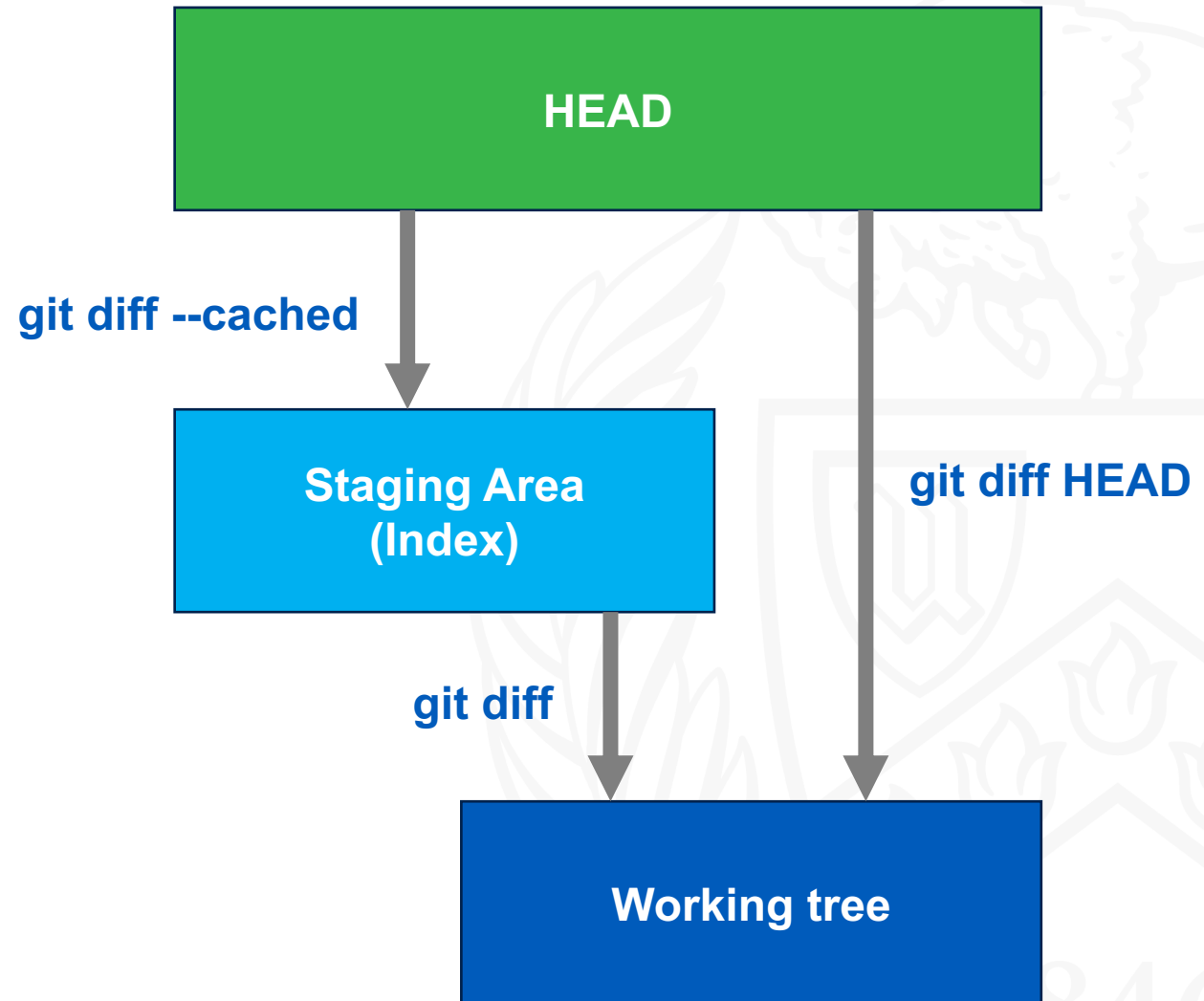
Force change, you will lose this file!

I will illustrate it, but do not follow

Viewing diffs of files

```
$ git diff  
$ git diff --cached  
$ git diff HEAD
```

```
$ git commit -a commit #3
```



The .gitignore file

- Specifies files that you don't want Git to track under version control
- Commonly used for compiled files, binaries, large asset files (e.g. images)
- Can use wildcards (e.g. *.pyc, *.png, Images/*, etc.)
- Be careful – if you add a file to .gitignore after it's already been tracked, potential issues
- A list of recommended .gitignore files:
<https://github.com/github/gitignore>



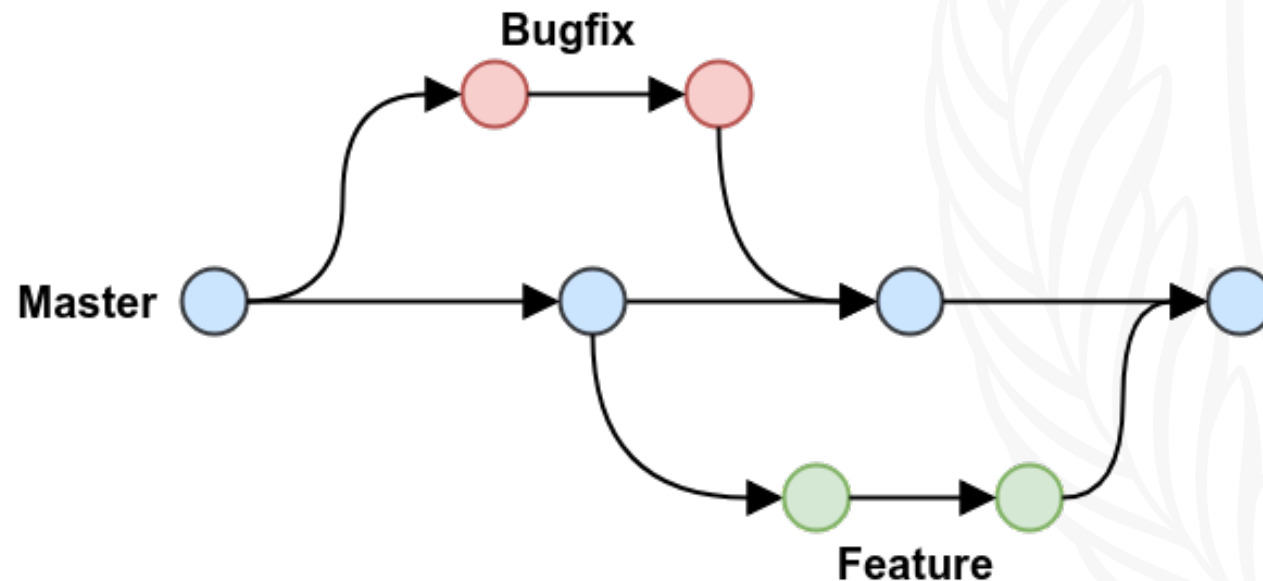
Use case #2: branching

- What is a branch?
- **Branching** commands
- The HEAD pointer
- Basics of **merging**
- Basics of **rebasing**
- Aside: the git reset command



What is a branch?

- Visualize a project's development as a “linked list” of commits.
- When a development track splits, a new branch is created.
- In Git, **branches are actually just a pointer to these commits**



Branching commands

- List all branches in the project – **git branch**
- Create a new branch – **git branch <branchname>**
- Switch to a branch – **git checkout <branchname>**
- Create and immediately switch – **git checkout -b <branchname>**
- Delete a branch – **git branch -d <branchname>**

```
$ git checkout -b new  
$ git branch  
$ git checkout master  
$ git branch
```

Stashing

- The working tree must be clean when switching branches
- Stash changes that you don't want to commit at that time – **git stash**
 - Puts a stash onto the stack
- Later, apply the most recent stashed changes and remove that stash – **git stash pop**

```
$ git checkout new  
$ vim b
```

```
$ git stash  
$ vim b  
$ git stash pop  
$ vim b  
$ git commit -a
```

This is another test??
Yifan loves chocolate ice cream

commit #1 on new branch

```
$ git checkout master  
$ vim b
```

This is another test??
Yifan loves strawberry ice cream

```
$ git commit -a  
commit #4
```